Task 1: Convolutional-Recurrent Architectures

Build a model based on convolutional-recurrent architectures for classifying Style, Artist, Genre, and other attributes. General and Specific. Pick the most appropriate approach and discuss your strategy. Use the ArtGAN dataset: https://github.com/cs-chan/ArtGAN/blob/master/WikiArt%20Dataset/README.md

Evaluation Metrics: discuss which evaluation metrics you are using to evaluate your model performance. Find outliers, e.g. paintings that do not fit a particular artist or genre despite their assignment.

```
# Imports
import os
import pandas as pd
import numpy as np
import random
import torch
import torch.nn as nn
from PIL import Image, ImageFile
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms
from torchvision.models import efficientnet_b3,
EfficientNet_B3_Weights
from tqdm import tqdm
import torch.nn.functional as F
from sklearn.metrics import f1_score, classification_report
import wandb

ImageFile.LOAD_TRUNCATED_IMAGES = True

# wandb login
from kaggle_secrets import UserSecretsClient
secrets = UserSecretsClient()
wandb.login(key=secrets.get_secret("WANDB_API_KEY"))
```

```
wandb: WARNING If you're specifying your api key in code, ensure this
code is not shared publicly.
wandb: WARNING Consider setting the WANDB_API_KEY environment
variable, or running `wandb login` from the command line.
wandb: [wandb.login()] Using explicit session credentials for
https://api.wandb.ai.
wandb: No netrc file found, creating one.
wandb: Appending key for api.wandb.ai to your netrc file: /root/.netrc
wandb: Currently logged in as: samanvithkashyap (samanvithkashyap-pes-
university) to https://api.wandb.ai. Use `wandb login --relogin` to
force relogin
```

```
True
```

1. Dataset Loader

PyTorch Dataset for the WikiArt Dataset. Returns the 3 labels for style, artist and genre. Handles missing or corrupted images

```
class WikiArtMultiTask(Dataset):

    def __init__(self, style_csv, artist_csv, genre_csv, img_dir,
transform=None):
        style_df = pd.read_csv(style_csv, header=None,
names=['filename', 'style'])
        artist_df = pd.read_csv(artist_csv, header=None,
names=['filename', 'artist'])
        genre_df = pd.read_csv(genre_csv, header=None,
names=['filename', 'genre'])

        merged = style_df.merge(artist_df, on='filename', how='inner')
        merged = merged.merge(genre_df, on='filename', how='inner')

        self.annotations = merged.reset_index(drop=True)
        self.img_dir = img_dir
        self.transform = transform
        print(f"Loaded {len(self.annotations)} images with all 3
labels "
              f"(style: {self.annotations['style'].nunique()}, "
              f"artist: {self.annotations['artist'].nunique()}, "
              f"genre: {self.annotations['genre'].nunique()})")

    def __len__(self):
        return len(self.annotations)

    def __getitem__(self, index):
        for attempt in range(min(10, len(self.annotations))):
            try:
                idx = (index + attempt) % len(self.annotations)
                row = self.annotations.iloc[idx]
                img_path = os.path.join(self.img_dir,
str(row['filename']))

                image = Image.open(img_path).convert("RGB")
                style_label = int(row['style'])
                artist_label = int(row['artist'])
                genre_label = int(row['genre'])

                if self.transform:
```

```
                    image = self.transform(image)
                return image, style_label, artist_label, genre_label,
img_path

            except Exception as e:
                continue
        raise RuntimeError(f"No images found from the dataset")
```

2. Model Architecture

Backbone: EfficientNet-B3 Task specific heads:

- **Style Head**: Column-wise reshape -> BiGRU (2 layers, hidden=256) -> FC -> 27 classes
- **Artist Head**: GAP -> FC(1536->512) -> ReLU -> Dropout(0.3) -> FC(512->195)
- **Genre Head**: GAP -> FC(1536->256) -> ReLU -> Dropout(0.3) -> FC(256->10)

Hierarchy: Genre (10) -> Style (27) -> Artist (195)

```
# Multi-Task CRNN Architecture
class ArtCRNN(nn.Module):
    def __init__(self, num_styles=27, num_artists=195, num_genres=10):
        super(ArtCRNN, self).__init__()

        # Shared Backbone: EfficientNet-B3
        basemodel =
efficientnet_b3(weights=EfficientNet_B3_Weights.DEFAULT)
        self.backbone = basemodel.features  # Output: (B, 1536, 8, 8)

        # GAP for artist/genre heads
        self.gap = nn.AdaptiveAvgPool2d(1)

        # Style Head: BiGRU on column-wise features
        self.style_rnn_input_dim = 1536 * 8  # channels * height =
12288
        self.style_hidden_size = 256
        self.style_rnn = nn.GRU(
            input_size=self.style_rnn_input_dim,
            hidden_size=self.style_hidden_size,
            num_layers=2,
            batch_first=True,
            bidirectional=True
        )
        self.style_fc = nn.Linear(self.style_hidden_size * 2,
num_styles)  # BiGRU -> 512 -> 27

        # Artist Head: FC with Dropout
        self.artist_head = nn.Sequential(
```

```
            nn.Linear(1536, 512),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(512, num_artists)
        )

        # Genre Head: FC with Dropout
        self.genre_head = nn.Sequential(
            nn.Linear(1536, 256),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, num_genres)
        )

    def forward(self, x):
        # Shared feature extraction
        features = self.backbone(x)  # (B, 1536, 8, 8)
        batch_size, channels, height, width = features.size()

        # --- Style: column-wise reshape -> BiGRU ---
        style_in = features.permute(0, 3, 1, 2).contiguous()  # (B, W,
C, H)
        style_in = style_in.view(batch_size, width, channels * height)
# (B, 8, 12288)
        _, style_hidden = self.style_rnn(style_in)
        # Concat last hidden from both directions
        style_out = torch.cat((style_hidden[-2], style_hidden[-1]),
dim=1)  # (B, 512)
        style_logits = self.style_fc(style_out)

        # --- Artist and Genre: GAP -> FC ---
        pooled = self.gap(features).view(batch_size, -1)  # (B, 1536)
        artist_logits = self.artist_head(pooled)
        genre_logits = self.genre_head(pooled)

        return style_logits, artist_logits, genre_logits
```

3. Training

Phase 1 (10 epochs): Freeze backbone, train only task heads at lr=1e-3
Phase 2 (up to 20 epochs): Unfreeze last 3 backbone blocks, fine-tune at lr=1e-4

- Early stopping: Stops if validation loss doesn't improve for 5 consecutive epochs
- Best model checkpoint is restored at the end

Multi-Task Loss: `L = Ls*L_style + La*L_artist + Lg*L_genre`

- Style: Weighted CrossEntropy (class imbalance), Ls=1.0
- Artist: CrossEntropy + Label Smoothing e=0.1 (subjective boundaries), La=0.8
- Genre: Weighted CrossEntropy (class imbalance), Lg=0.8

Mixed precision (fp16) + CosineAnnealingLR scheduler per phase. Validation metrics logged after every epoch to wandb for monitoring.

```
def compute_class_weights(csv_path, label_col, num_classes, device):
    """Compute inverse-frequency weights for imbalanced classes."""
    df = pd.read_csv(csv_path, header=None)
    counts = df[label_col].value_counts().sort_index()
    weights = 1.0 / counts.values.astype(np.float32)
    weights = weights / weights.sum() * num_classes  # normalize
    return torch.tensor(weights, dtype=torch.float32).to(device)


def freeze_backbone(model):
    """Freeze all backbone parameters."""
    for param in model.backbone.parameters():
        param.requires_grad = False


def unfreeze_last_n_blocks(model, n=3):
    """Unfreeze the last n blocks of EfficientNet backbone."""
    # EfficientNet-B3 has 8 blocks (indices 0-7) in .backbone
    blocks = list(model.backbone.children())
    # Unfreeze last n blocks
    for block in blocks[-n:]:
        for param in block.parameters():
            param.requires_grad = True


def evaluate_model(model, val_loader, device, style_criterion,
artist_criterion, genre_criterion):
    """Evaluate model on validation set and return metrics."""
    model.eval()
    val_loss = {'style': 0.0, 'artist': 0.0, 'genre': 0.0, 'total':
0.0}
    correct = {'style': 0, 'artist': 0, 'genre': 0}
    total = 0

    with torch.no_grad():
        for images, style_labels, artist_labels, genre_labels, _ in
val_loader:
            images = images.to(device)
            style_labels = style_labels.to(device)
            artist_labels = artist_labels.to(device)
            genre_labels = genre_labels.to(device)
```

```
            style_out, artist_out, genre_out = model(images)

            loss_style = style_criterion(style_out, style_labels)
            loss_artist = artist_criterion(artist_out, artist_labels)
            loss_genre = genre_criterion(genre_out, genre_labels)
            total_loss = loss_style + 0.8 * loss_artist + 0.8 *
loss_genre

            val_loss['style'] += loss_style.item()
            val_loss['artist'] += loss_artist.item()
            val_loss['genre'] += loss_genre.item()
            val_loss['total'] += total_loss.item()

            _, style_pred = torch.max(style_out, 1)
            _, artist_pred = torch.max(artist_out, 1)
            _, genre_pred = torch.max(genre_out, 1)

            total += style_labels.size(0)
            correct['style'] += (style_pred ==
style_labels).sum().item()
            correct['artist'] += (artist_pred ==
artist_labels).sum().item()
            correct['genre'] += (genre_pred ==
genre_labels).sum().item()

    n_batches = len(val_loader)
    avg_loss = {k: v / n_batches for k, v in val_loss.items()}
    acc = {k: 100.0 * v / total for k, v in correct.items()}

    return avg_loss, acc


def train_model(model, train_loader, val_loader, device,
                style_weights, genre_weights,
                epochs_phase1=10, epochs_phase2=20,
                lambda_style=1.0, lambda_artist=0.8, lambda_genre=0.8,
                early_stop_patience=5):
    """Two-phase multi-task training with validation monitoring and
early stopping."""

    # Loss functions
    style_criterion = nn.CrossEntropyLoss(weight=style_weights)
    artist_criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
    genre_criterion = nn.CrossEntropyLoss(weight=genre_weights)

    # For validation (no class weights to get unbiased estimate)
    val_style_criterion = nn.CrossEntropyLoss()
    val_artist_criterion = nn.CrossEntropyLoss()
```

```
    val_genre_criterion = nn.CrossEntropyLoss()

    scaler = torch.amp.GradScaler('cuda')
    best_val_loss = float('inf')
    best_model_state = None

    # ==================== PHASE 1: Frozen Backbone
====================
    print("=" * 60)
    print("PHASE 1: Training heads only (backbone frozen)")
    print("=" * 60)

    freeze_backbone(model)
    head_params = [p for p in model.parameters() if p.requires_grad]
    optimizer = optim.AdamW(head_params, lr=1e-3)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer,
T_max=epochs_phase1)

    for epoch in range(epochs_phase1):
        model.train()
        running_loss = {'style': 0.0, 'artist': 0.0, 'genre': 0.0,
'total': 0.0}
        progress_bar = tqdm(train_loader, desc=f"P1 Epoch
{epoch+1}/{epochs_phase1}")

        for images, style_labels, artist_labels, genre_labels, _ in
progress_bar:
            images = images.to(device)
            style_labels = style_labels.to(device)
            artist_labels = artist_labels.to(device)
            genre_labels = genre_labels.to(device)

            optimizer.zero_grad()

            with torch.amp.autocast('cuda'):
                style_out, artist_out, genre_out = model(images)

                loss_style = style_criterion(style_out, style_labels)
                loss_artist = artist_criterion(artist_out,
artist_labels)
                loss_genre = genre_criterion(genre_out, genre_labels)

                total_loss = (lambda_style * loss_style +
                              lambda_artist * loss_artist +
                              lambda_genre * loss_genre)

            scaler.scale(total_loss).backward()
            scaler.step(optimizer)
            scaler.update()
```

```
            running_loss['style'] += loss_style.item()
            running_loss['artist'] += loss_artist.item()
            running_loss['genre'] += loss_genre.item()
            running_loss['total'] += total_loss.item()

            progress_bar.set_postfix(
                total=f"{total_loss.item():.3f}",
                s=f"{loss_style.item():.3f}",
                a=f"{loss_artist.item():.3f}",
                g=f"{loss_genre.item():.3f}"
            )

        scheduler.step()
        n_batches = len(train_loader)
        avg = {k: v / n_batches for k, v in running_loss.items()}

        # Validation
        val_loss, val_acc = evaluate_model(model, val_loader, device,
                                           val_style_criterion,
val_artist_criterion, val_genre_criterion)

        print(f"P1 Epoch {epoch+1} -- Train: {avg['total']:.4f} | Val:
{val_loss['total']:.4f} | "
              f"Val Acc: S={val_acc['style']:.1f}%
A={val_acc['artist']:.1f}% G={val_acc['genre']:.1f}%")

        wandb.log({
            "phase": 1, "epoch": epoch + 1,
            "train_loss_total": avg['total'],
            "train_loss_style": avg['style'],
            "train_loss_artist": avg['artist'],
            "train_loss_genre": avg['genre'],
            "val_loss_total": val_loss['total'],
            "val_loss_style": val_loss['style'],
            "val_loss_artist": val_loss['artist'],
            "val_loss_genre": val_loss['genre'],
            "val_acc_style": val_acc['style'],
            "val_acc_artist": val_acc['artist'],
            "val_acc_genre": val_acc['genre'],
            "lr": scheduler.get_last_lr()[0]
        })

    # ==================== PHASE 2: Fine-tune Last 3 Blocks
====================
    print("\n" + "=" * 60)
    print("PHASE 2: Fine-tuning last 3 backbone blocks (with early
stopping)")
    print("=" * 60)
```

```
    unfreeze_last_n_blocks(model, n=3)
    optimizer = optim.AdamW(filter(lambda p: p.requires_grad,
model.parameters()), lr=1e-4)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer,
T_max=epochs_phase2)

    patience_counter = 0

    for epoch in range(epochs_phase2):
        model.train()
        running_loss = {'style': 0.0, 'artist': 0.0, 'genre': 0.0,
'total': 0.0}
        progress_bar = tqdm(train_loader, desc=f"P2 Epoch
{epoch+1}/{epochs_phase2}")

        for images, style_labels, artist_labels, genre_labels, _ in
progress_bar:
            images = images.to(device)
            style_labels = style_labels.to(device)
            artist_labels = artist_labels.to(device)
            genre_labels = genre_labels.to(device)

            optimizer.zero_grad()

            with torch.amp.autocast('cuda'):
                style_out, artist_out, genre_out = model(images)

                loss_style = style_criterion(style_out, style_labels)
                loss_artist = artist_criterion(artist_out,
artist_labels)
                loss_genre = genre_criterion(genre_out, genre_labels)

                total_loss = (lambda_style * loss_style +
                              lambda_artist * loss_artist +
                              lambda_genre * loss_genre)

            scaler.scale(total_loss).backward()
            scaler.step(optimizer)
            scaler.update()

            running_loss['style'] += loss_style.item()
            running_loss['artist'] += loss_artist.item()
            running_loss['genre'] += loss_genre.item()
            running_loss['total'] += total_loss.item()

            progress_bar.set_postfix(
                total=f"{total_loss.item():.3f}",
                s=f"{loss_style.item():.3f}",
```

```
                a=f"{loss_artist.item():.3f}",
                g=f"{loss_genre.item():.3f}"
            )

        scheduler.step()
        n_batches = len(train_loader)
        avg = {k: v / n_batches for k, v in running_loss.items()}

        # Validation
        val_loss, val_acc = evaluate_model(model, val_loader, device,
                                           val_style_criterion,
val_artist_criterion, val_genre_criterion)

        print(f"P2 Epoch {epoch+1} -- Train: {avg['total']:.4f} | Val:
{val_loss['total']:.4f} | "
              f"Val Acc: S={val_acc['style']:.1f}%
A={val_acc['artist']:.1f}% G={val_acc['genre']:.1f}%")

        wandb.log({
            "phase": 2, "epoch": epochs_phase1 + epoch + 1,
            "train_loss_total": avg['total'],
            "train_loss_style": avg['style'],
            "train_loss_artist": avg['artist'],
            "train_loss_genre": avg['genre'],
            "val_loss_total": val_loss['total'],
            "val_loss_style": val_loss['style'],
            "val_loss_artist": val_loss['artist'],
            "val_loss_genre": val_loss['genre'],
            "val_acc_style": val_acc['style'],
            "val_acc_artist": val_acc['artist'],
            "val_acc_genre": val_acc['genre'],
            "lr": scheduler.get_last_lr()[0]
        })

        # Early stopping check
        if val_loss['total'] < best_val_loss:
            best_val_loss = val_loss['total']
            best_model_state = model.state_dict().copy()
            patience_counter = 0
            print(f"  -> New best model! Val loss:
{best_val_loss:.4f}")
        else:
            patience_counter += 1
            print(f"  -> No improvement
({patience_counter}/{early_stop_patience})")

            if patience_counter >= early_stop_patience:
                print(f"\nEarly stopping triggered at epoch
{epoch+1}!")
```

```
                break

    # Restore best model
    if best_model_state is not None:
        model.load_state_dict(best_model_state)
        print(f"\nRestored best model with val loss:
{best_val_loss:.4f}")
```

4. Finding the outliers

Identifies paintings with high multi task loss. An image confused across multiple head is a stronger outlier signal than a single task mistake.

Uses pre-image loss from all three heads.

```
def find_outliers(model, val_loader, device, top_k=10):
    model.eval()
    criterion = nn.CrossEntropyLoss(reduction='none')
    outlier_data = []
    seen_paths = set()

    with torch.no_grad():
        for images, style_labels, artist_labels, genre_labels, paths
in tqdm(val_loader, desc="Hunting Outliers"):
            images = images.to(device)
            style_labels = style_labels.to(device)
            artist_labels = artist_labels.to(device)
            genre_labels = genre_labels.to(device)

            style_out, artist_out, genre_out = model(images)

            style_loss = criterion(style_out, style_labels)
            artist_loss = criterion(artist_out, artist_labels)
            genre_loss = criterion(genre_out, genre_labels)
            combined_loss = style_loss + artist_loss + genre_loss

            style_pred = torch.argmax(style_out, dim=1)
            artist_pred = torch.argmax(artist_out, dim=1)
            genre_pred = torch.argmax(genre_out, dim=1)

            for i in range(len(images)):
                path = paths[i]
                if path not in seen_paths:
                    seen_paths.add(path)
                    outlier_data.append({
                        'image_path': path,
                        'true_style': style_labels[i].item(),
```

```
                        'pred_style': style_pred[i].item(),
                        'style_loss': style_loss[i].item(),
                        'true_artist': artist_labels[i].item(),
                        'pred_artist': artist_pred[i].item(),
                        'artist_loss': artist_loss[i].item(),
                        'true_genre': genre_labels[i].item(),
                        'pred_genre': genre_pred[i].item(),
                        'genre_loss': genre_loss[i].item(),
                        'combined_loss': combined_loss[i].item(),
                    })

    df = pd.DataFrame(outlier_data)
    top_outliers = df.sort_values(by='combined_loss',
ascending=False).head(top_k)
    return top_outliers

# paths to csv and dataset
STYLE_TRAIN_CSV =
"/kaggle/input/datasets/samanvithkashyap/wikicsv/style_train.csv"
STYLE_VAL_CSV =
"/kaggle/input/datasets/samanvithkashyap/wikicsv/style_val.csv"
ARTIST_TRAIN_CSV =
"/kaggle/input/datasets/samanvithkashyap/wikicsv/artist_train.csv"
ARTIST_VAL_CSV =
"/kaggle/input/datasets/samanvithkashyap/wikicsv/artist_val.csv"
GENRE_TRAIN_CSV =
"/kaggle/input/datasets/samanvithkashyap/wikicsv/genre_train.csv"
GENRE_VAL_CSV =
"/kaggle/input/datasets/samanvithkashyap/wikicsv/genre_val.csv"
IMG_DIR = "/kaggle/input/datasets/steubk/wikiart"

# Hyperparameters
NUM_STYLES = 27
NUM_ARTISTS = 195
NUM_GENRES = 10
IMG_SIZE = 256
BATCH_SIZE = 32
NUM_WORKERS = 4
EPOCHS_PHASE1 = 10
EPOCHS_PHASE2 = 20
EARLY_STOP_PATIENCE = 5
LAMBDA_STYLE = 1.0
LAMBDA_ARTIST = 0.8
LAMBDA_GENRE = 0.8
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Initialize wandb
wandb.init(project="artextract-wikiart", name="multitask-crnn-b3",
config={
    "img_size": IMG_SIZE,
```

```
    "batch_size": BATCH_SIZE,
    "epochs_phase1": EPOCHS_PHASE1,
    "epochs_phase2": EPOCHS_PHASE2,
    "early_stop_patience": EARLY_STOP_PATIENCE,
    "num_styles": NUM_STYLES,
    "num_artists": NUM_ARTISTS,
    "num_genres": NUM_GENRES,
    "architecture": "EfficientNet-B3 + BiGRU (multi-task)",
    "optimizer": "AdamW",
    "lr_phase1": 1e-3,
    "lr_phase2": 1e-4,
    "lambda_style": LAMBDA_STYLE,
    "lambda_artist": LAMBDA_ARTIST,
    "lambda_genre": LAMBDA_GENRE,
    "label_smoothing_artist": 0.1,
})

# Augmentation for training
train_transforms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.2, contrast=0.2,
saturation=0.1),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
0.224, 0.225])
])

val_transforms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
0.224, 0.225])
])

# Datasets & DataLoaders
train_dataset = WikiArtMultiTask(
    style_csv=STYLE_TRAIN_CSV, artist_csv=ARTIST_TRAIN_CSV,
genre_csv=GENRE_TRAIN_CSV,
    img_dir=IMG_DIR, transform=train_transforms
)
val_dataset = WikiArtMultiTask(
    style_csv=STYLE_VAL_CSV, artist_csv=ARTIST_VAL_CSV,
genre_csv=GENRE_VAL_CSV,
    img_dir=IMG_DIR, transform=val_transforms
)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE,
shuffle=True,
```

```
                            num_workers=NUM_WORKERS, pin_memory=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE,
shuffle=False,
                        num_workers=NUM_WORKERS, pin_memory=True)

# Class weights for imbalanced tasks
style_weights = compute_class_weights(STYLE_TRAIN_CSV, 1, NUM_STYLES,
device)
genre_weights = compute_class_weights(GENRE_TRAIN_CSV, 1, NUM_GENRES,
device)

# Model
model = ArtCRNN(num_styles=NUM_STYLES, num_artists=NUM_ARTISTS,
num_genres=NUM_GENRES).to(device)
wandb.watch(model, log="gradients", log_freq=100)

# Train
# === Model ===
model = ArtCRNN(num_styles=NUM_STYLES, num_artists=NUM_ARTISTS,
num_genres=NUM_GENRES).to(device)
wandb.watch(model, log="gradients", log_freq=100)

# === Train ===
print("Starting Multi-Task Training")
train_model(model, train_loader, val_loader, device,
            style_weights=style_weights, genre_weights=genre_weights,
            epochs_phase1=EPOCHS_PHASE1, epochs_phase2=EPOCHS_PHASE2,
            lambda_style=LAMBDA_STYLE, lambda_artist=LAMBDA_ARTIST,
lambda_genre=LAMBDA_GENRE,
            early_stop_patience=EARLY_STOP_PATIENCE)

torch.save(model.state_dict(), "artcrnn_multitask_weights.pth")
print("Model saved: artcrnn_multitask_weights.pth")

# === Outlier Detection ===
top_10 = find_outliers(model, val_loader, device)
top_10.to_csv("top_10_outliers.csv", index=False)

# ==================== EVALUATION ====================
print("\n" + "=" * 60)
print("EVALUATION")
print("=" * 60)

model.eval()
all_style_labels, all_style_preds = [], []
all_artist_labels, all_artist_preds = [], []
all_genre_labels, all_genre_preds = [], []
correct = {'style': 0, 'artist': 0, 'genre': 0, 'artist_top5': 0}
total = 0
```

```
with torch.no_grad():
    for images, style_labels, artist_labels, genre_labels, _ in
tqdm(val_loader, desc="Evaluating"):
        images = images.to(device)
        style_labels = style_labels.to(device)
        artist_labels = artist_labels.to(device)
        genre_labels = genre_labels.to(device)

        style_out, artist_out, genre_out = model(images)

        # Top-1 predictions
        _, style_pred = torch.max(style_out, 1)
        _, artist_pred = torch.max(artist_out, 1)
        _, genre_pred = torch.max(genre_out, 1)

        # Top-5 for artist
        _, artist_top5 = artist_out.topk(5, dim=1)
        artist_top5_correct =
artist_top5.eq(artist_labels.unsqueeze(1)).any(dim=1)

        total += style_labels.size(0)
        correct['style'] += (style_pred == style_labels).sum().item()
        correct['artist'] += (artist_pred ==
artist_labels).sum().item()
        correct['genre'] += (genre_pred == genre_labels).sum().item()
        correct['artist_top5'] += artist_top5_correct.sum().item()

        all_style_labels.extend(style_labels.cpu().numpy())
        all_style_preds.extend(style_pred.cpu().numpy())
        all_artist_labels.extend(artist_labels.cpu().numpy())
        all_artist_preds.extend(artist_pred.cpu().numpy())
        all_genre_labels.extend(genre_labels.cpu().numpy())
        all_genre_preds.extend(genre_pred.cpu().numpy())

# === F1 Scores ===
style_f1 = f1_score(all_style_labels, all_style_preds,
average='weighted')
artist_f1 = f1_score(all_artist_labels, all_artist_preds,
average='weighted')
genre_f1 = f1_score(all_genre_labels, all_genre_preds,
average='weighted')
avg_f1 = (style_f1 + artist_f1 + genre_f1) / 3.0

# === Accuracies ===
style_acc = 100 * correct['style'] / total
artist_acc = 100 * correct['artist'] / total
artist_top5_acc = 100 * correct['artist_top5'] / total
genre_acc = 100 * correct['genre'] / total

print(f"\n{'Task':<12} {'Accuracy':>10} {'Weighted F1':>12}")
```

```
print("-" * 36)
print(f"{'Style':<12} {style_acc:>9.2f}% {style_f1:>11.4f}")
print(f"{'Artist':<12} {artist_acc:>9.2f}% {artist_f1:>11.4f}")
print(f"{'Artist@5':<12} {artist_top5_acc:>9.2f}%")
print(f"{'Genre':<12} {genre_acc:>9.2f}% {genre_f1:>11.4f}")
print("-" * 36)
print(f"{'AVG F1':<12} {'':>10} {avg_f1:>11.4f}")

# === Per-class reports ===
style_names = sorted([d for d in os.listdir(IMG_DIR)
                      if os.path.isdir(os.path.join(IMG_DIR, d))])

# Get unique labels present in validation data
unique_style_labels = sorted(set(all_style_labels) |
set(all_style_preds))
unique_genre_labels = sorted(set(all_genre_labels) |
set(all_genre_preds))

print(f"\n--- Style Classification Report ({len(unique_style_labels)}
classes present in validation) ---")
print(classification_report(all_style_labels, all_style_preds,
                            labels=unique_style_labels,
                            target_names=[style_names[i] for i in
unique_style_labels],
                            digits=4, zero_division=0))

print(f"\n--- Genre Classification Report ({len(unique_genre_labels)}
classes present in validation) ---")
print(classification_report(all_genre_labels, all_genre_preds,
                            labels=unique_genre_labels, digits=4,
zero_division=0))

# === Log everything to wandb ===
wandb.log({
    "val_style_acc": style_acc,
    "val_style_f1": style_f1,
    "val_artist_acc": artist_acc,
    "val_artist_top5_acc": artist_top5_acc,
    "val_artist_f1": artist_f1,
    "val_genre_acc": genre_acc,
    "val_genre_f1": genre_f1,
    "val_avg_f1": avg_f1,
})

# Per-class style F1 to wandb
style_f1_per_class = f1_score(all_style_labels, all_style_preds,
labels=unique_style_labels, average=None, zero_division=0)
for i, label_idx in enumerate(unique_style_labels):
    wandb.log({f"f1_style_{style_names[label_idx]}":
style_f1_per_class[i]})
```

```
wandb.finish()
```

```
wandb: Tracking run with wandb version 0.24.0
wandb: Run data is saved locally in /kaggle/working/wandb/run-
20260311_011902-9yweqlsn
wandb: Run `wandb offline` to turn off syncing.
wandb: Syncing run multitask-crnn-b3
wandb:  View project at https://wandb.ai/samanvithkashyap-pes-
university/artextract-wikiart
wandb: 🚀 View run at https://wandb.ai/samanvithkashyap-pes-
university/artextract-wikiart/runs/9yweqlsn

Loaded 11276 images with all 3 labels (style: 16, artist: 23, genre:
10)
Loaded 4707 images with all 3 labels (style: 16, artist: 23, genre:
10)
Downloading:
"https://download.pytorch.org/models/efficientnet_b3_rwightman-
b3899882.pth" to
/root/.cache/torch/hub/checkpoints/efficientnet_b3_rwightman-
b3899882.pth

100%|██████████| 47.2M/47.2M [00:00<00:00, 179MB/s]

Starting Multi-Task Training
============================================================
PHASE 1: Training heads only (backbone frozen)
============================================================

P1 Epoch 1/10: 100%|██████████| 353/353 [03:23<00:00,  1.73it/s,
a=2.040, g=0.925, s=0.764, total=3.135]

P1 Epoch 1 -- Train: 4.5495 | Val: 3.0578 | Val Acc: S=63.8% A=62.9%
G=64.1%

P1 Epoch 2/10: 100%|██████████| 353/353 [02:37<00:00,  2.24it/s,
a=1.661, g=1.032, s=0.842, total=2.996]

P1 Epoch 2 -- Train: 3.4254 | Val: 2.8846 | Val Acc: S=66.0% A=65.9%
G=63.4%

P1 Epoch 3/10: 100%|██████████| 353/353 [02:38<00:00,  2.23it/s,
a=1.541, g=1.505, s=0.875, total=3.312]

P1 Epoch 3 -- Train: 3.1382 | Val: 2.7530 | Val Acc: S=66.6% A=68.3%
G=66.5%

P1 Epoch 4/10: 100%|██████████| 353/353 [02:35<00:00,  2.27it/s,
a=1.765, g=0.894, s=0.536, total=2.663]
```

```
P1 Epoch 4 -- Train: 2.9251 | Val: 2.6531 | Val Acc: S=67.2% A=69.2%
G=67.1%

P1 Epoch 5/10: 100%|██████████| 353/353 [02:35<00:00,  2.28it/s,
a=1.776, g=0.791, s=0.263, total=2.316]

P1 Epoch 5 -- Train: 2.7071 | Val: 2.6310 | Val Acc: S=67.0% A=70.7%
G=67.2%

P1 Epoch 6/10: 100%|██████████| 353/353 [02:35<00:00,  2.27it/s,
a=1.959, g=0.302, s=0.735, total=2.544]

P1 Epoch 6 -- Train: 2.5784 | Val: 2.5419 | Val Acc: S=68.9% A=71.2%
G=68.8%

P1 Epoch 7/10: 100%|██████████| 353/353 [02:34<00:00,  2.28it/s,
a=2.035, g=0.590, s=0.062, total=2.162]

P1 Epoch 7 -- Train: 2.4254 | Val: 2.5354 | Val Acc: S=68.0% A=72.4%
G=68.7%

P1 Epoch 8/10: 100%|██████████| 353/353 [02:36<00:00,  2.25it/s,
a=1.605, g=0.813, s=0.557, total=2.491]

P1 Epoch 8 -- Train: 2.3452 | Val: 2.4779 | Val Acc: S=68.9% A=72.5%
G=68.8%

P1 Epoch 9/10: 100%|██████████| 353/353 [02:36<00:00,  2.25it/s,
a=1.816, g=0.875, s=0.546, total=2.699]

P1 Epoch 9 -- Train: 2.2667 | Val: 2.4654 | Val Acc: S=70.2% A=73.0%
G=69.4%

P1 Epoch 10/10: 100%|██████████| 353/353 [02:37<00:00,  2.23it/s,
a=1.770, g=0.749, s=0.670, total=2.686]

P1 Epoch 10 -- Train: 2.2464 | Val: 2.4861 | Val Acc: S=69.2% A=72.6%
G=68.4%

============================================================
PHASE 2: Fine-tuning last 3 backbone blocks (with early stopping)
============================================================

P2 Epoch 1/20: 100%|██████████| 353/353 [02:43<00:00,  2.16it/s,
a=1.572, g=0.774, s=0.094, total=1.971]

P2 Epoch 1 -- Train: 2.1648 | Val: 2.2108 | Val Acc: S=73.2% A=76.5%
G=71.1%
  -> New best model! Val loss: 2.2108

P2 Epoch 2/20: 100%|██████████| 353/353 [02:40<00:00,  2.20it/s,
a=1.172, g=0.312, s=0.131, total=1.318]
```

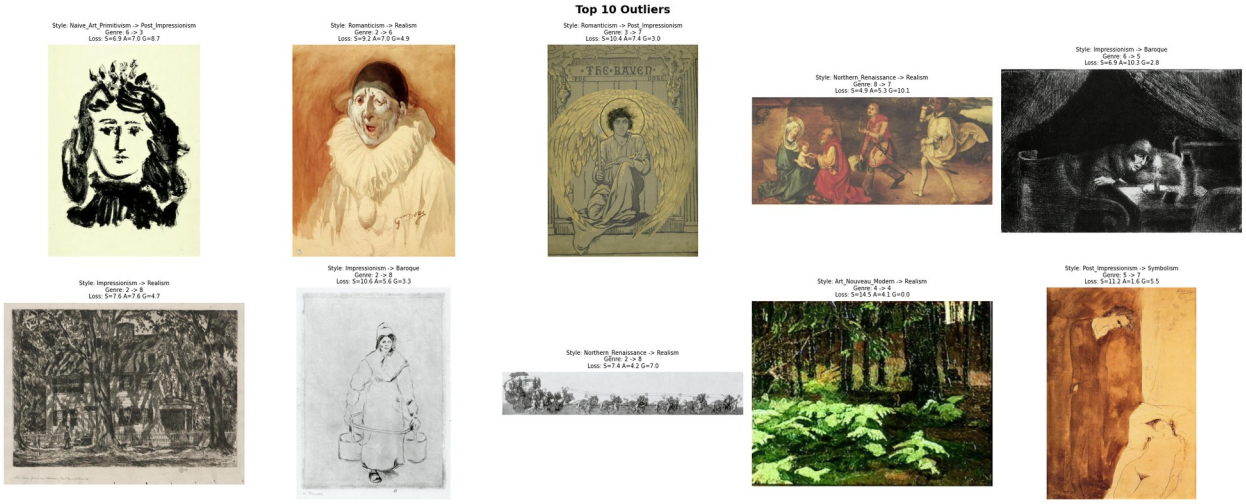
```
P2 Epoch 2 -- Train: 1.9623 | Val: 2.0992 | Val Acc: S=74.8% A=78.7%
G=72.2%
  -> New best model! Val loss: 2.0992

P2 Epoch 3/20: 100%|██████████| 353/353 [02:40<00:00,  2.19it/s,
a=1.376, g=0.963, s=0.492, total=2.364]

P2 Epoch 3 -- Train: 1.8235 | Val: 2.0795 | Val Acc: S=75.1% A=79.2%
G=72.3%
  -> New best model! Val loss: 2.0795

P2 Epoch 4/20: 100%|██████████| 353/353 [02:40<00:00,  2.20it/s,
a=1.395, g=0.694, s=0.319, total=1.991]

P2 Epoch 4 -- Train: 1.7123 | Val: 2.0077 | Val Acc: S=76.0% A=80.4%
G=73.0%
  -> New best model! Val loss: 2.0077

P2 Epoch 5/20: 100%|██████████| 353/353 [02:38<00:00,  2.23it/s,
a=1.571, g=0.557, s=0.106, total=1.808]

P2 Epoch 5 -- Train: 1.6263 | Val: 1.9658 | Val Acc: S=76.7% A=81.3%
G=74.6%
  -> New best model! Val loss: 1.9658

P2 Epoch 6/20: 100%|██████████| 353/353 [02:39<00:00,  2.21it/s,
a=1.252, g=0.654, s=0.234, total=1.758]

P2 Epoch 6 -- Train: 1.5350 | Val: 1.9732 | Val Acc: S=75.7% A=81.5%
G=75.0%
  -> No improvement (1/5)

P2 Epoch 7/20: 100%|██████████| 353/353 [02:39<00:00,  2.22it/s,
a=1.141, g=0.593, s=0.053, total=1.441]

P2 Epoch 7 -- Train: 1.4537 | Val: 1.9292 | Val Acc: S=77.6% A=82.1%
G=74.7%
  -> New best model! Val loss: 1.9292

P2 Epoch 8/20: 100%|██████████| 353/353 [02:40<00:00,  2.20it/s,
a=1.091, g=0.599, s=0.087, total=1.439]

P2 Epoch 8 -- Train: 1.3933 | Val: 1.9543 | Val Acc: S=77.3% A=82.0%
G=75.6%
  -> No improvement (1/5)

P2 Epoch 9/20: 100%|██████████| 353/353 [02:39<00:00,  2.22it/s,
a=1.302, g=0.753, s=0.469, total=2.112]

P2 Epoch 9 -- Train: 1.3709 | Val: 1.9447 | Val Acc: S=77.0% A=82.9%
G=76.1%
  -> No improvement (2/5)
```

```
P2 Epoch 10/20: 100%|██████████| 353/353 [02:39<00:00,  2.22it/s,
a=1.492, g=0.443, s=0.229, total=1.777]

P2 Epoch 10 -- Train: 1.3092 | Val: 1.9127 | Val Acc: S=78.4% A=82.9%
G=76.4%
  -> New best model! Val loss: 1.9127

P2 Epoch 11/20: 100%|██████████| 353/353 [02:41<00:00,  2.19it/s,
a=1.109, g=0.543, s=0.173, total=1.495]

P2 Epoch 11 -- Train: 1.2643 | Val: 1.9080 | Val Acc: S=77.9% A=83.0%
G=76.0%
  -> New best model! Val loss: 1.9080

P2 Epoch 12/20: 100%|██████████| 353/353 [02:38<00:00,  2.23it/s,
a=1.450, g=0.192, s=0.214, total=1.527]

P2 Epoch 12 -- Train: 1.2326 | Val: 1.9292 | Val Acc: S=78.7% A=83.4%
G=76.3%
  -> No improvement (1/5)

P2 Epoch 13/20: 100%|██████████| 353/353 [02:37<00:00,  2.24it/s,
a=1.226, g=0.195, s=0.167, total=1.304]

P2 Epoch 13 -- Train: 1.2091 | Val: 1.9192 | Val Acc: S=78.3% A=83.6%
G=76.7%
  -> No improvement (2/5)

P2 Epoch 14/20: 100%|██████████| 353/353 [02:40<00:00,  2.20it/s,
a=1.292, g=0.436, s=0.066, total=1.449]

P2 Epoch 14 -- Train: 1.1873 | Val: 1.9395 | Val Acc: S=78.3% A=83.6%
G=77.2%
  -> No improvement (3/5)

P2 Epoch 15/20: 100%|██████████| 353/353 [02:38<00:00,  2.22it/s,
a=1.296, g=0.740, s=0.076, total=1.704]

P2 Epoch 15 -- Train: 1.1728 | Val: 1.9304 | Val Acc: S=78.6% A=83.7%
G=77.1%
  -> No improvement (4/5)

P2 Epoch 16/20: 100%|██████████| 353/353 [02:38<00:00,  2.23it/s,
a=1.256, g=0.456, s=0.162, total=1.532]

P2 Epoch 16 -- Train: 1.1467 | Val: 1.9197 | Val Acc: S=78.6% A=83.5%
G=77.1%
  -> No improvement (5/5)

Early stopping triggered at epoch 16!
```

```
Restored best model with val loss: 1.9080
Model saved: artcrnn_multitask_weights.pth

Hunting Outliers: 100%|██████████| 148/148 [01:02<00:00,  2.36it/s]


============================================================
EVALUATION
============================================================

Evaluating: 100%|██████████| 148/148 [01:02<00:00,  2.37it/s]
wandb: updating run metadata


Task           Accuracy  Weighted F1
------------------------------------
Style            78.65%      0.7870
Artist           83.51%      0.8333
Artist@5         96.98%
Genre            77.08%      0.7708
------------------------------------
AVG F1                       0.7970

--- Style Classification Report (16 classes present in validation) ---
                         precision    recall  f1-score   support

Abstract_Expressionism     0.0000    0.0000    0.0000         4
     Analytical_Cubism     0.6250    0.7143    0.6667        14
    Art_Nouveau_Modern     0.7200    0.6626    0.6901       163
               Baroque     0.8544    0.9167    0.8844       192
                Cubism     0.6218    0.6271    0.6245       118
         Expressionism     0.4096    0.4302    0.4196       179
               Fauvism     0.0000    0.0000    0.0000        11
         Impressionism     0.8887    0.8759    0.8822      1458
 Naive_Art_Primitivism     0.8272    0.8061    0.8165       196
  Northern_Renaissance     0.8883    0.8714    0.8798       210
           Pointillism     0.3333    0.1429    0.2000         7
    Post_Impressionism     0.5837    0.6477    0.6140       474
               Realism     0.7380    0.7500    0.7439       980
            Romanticism     0.9327    0.8445    0.8864       328
              Symbolism     0.8754    0.8560    0.8655       361
      Synthetic_Cubism     0.5556    0.8333    0.6667        12

              accuracy                         0.7865      4707
             macro avg     0.6158    0.6237    0.6150      4707
          weighted avg     0.7886    0.7865    0.7870      4707


--- Genre Classification Report (10 classes present in validation) ---
               precision    recall  f1-score   support
```

```
           0     0.0000    0.0000    0.0000         1
           1     0.6795    0.8080    0.7382       349
           2     0.6813    0.5809    0.6271       773
           3     0.7415    0.8454    0.7901       207
           4     0.8837    0.8409    0.8618      1301
           5     0.5734    0.7069    0.6332       116
           6     0.8697    0.7957    0.8311       847
           7     0.7389    0.6821    0.7093       390
           8     0.6557    0.8141    0.7264       538
           9     0.7887    0.9081    0.8442       185

    accuracy                         0.7708      4707
   macro avg     0.6612    0.6982    0.6761      4707
weighted avg     0.7769    0.7708    0.7708      4707


wandb: uploading config.yaml
wandb:
wandb: Run history:
wandb:                            epoch ▁▁▁
```

```
other file(s)
wandb: Find logs at: ./wandb/run-20260311_011902-9yweqlsn/logs

import matplotlib.pyplot as plt
from PIL import Image

fig, axes = plt.subplots(2, 5, figsize=(22, 9))
axes = axes.flatten()

style_names = sorted([d for d in
os.listdir('/kaggle/input/datasets/steubk/wikiart')
                        if
os.path.isdir(os.path.join('/kaggle/input/datasets/steubk/wikiart',
d))])

for idx, (_, row) in enumerate(top_10.iterrows()):
    img_path = os.path.join('/kaggle/input/datasets/steubk/wikiart',
row['image_path'])
    img = Image.open(img_path)
    axes[idx].imshow(img)

    title = (f"Style: {style_names[row['true_style']]} ->
{style_names[row['pred_style']]}\n"
             f"Genre: {row['true_genre']} -> {row['pred_genre']}\n"
             f"Loss: S={row['style_loss']:.1f}
A={row['artist_loss']:.1f} G={row['genre_loss']:.1f}")

    axes[idx].set_title(title, fontsize=7)
    axes[idx].axis('off')

plt.suptitle('Top 10 Outliers',
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('outliers_multitask.png', dpi=150, bbox_inches='tight')
plt.show()
```



5. Strategy Discussion and Results

Architecture Choice: Why CRNN over pure CNN or ViT?

- **Pure CNN** (ResNet, EfficientNet + FC): Treats the image as a bag of features. Loses spatial ordering of brushstrokes -- critical for distinguishing Impressionism from Post-Impressionism where technique differs but palette overlaps.
- **Vision Transformer**: Requires large datasets (>300k) to outperform CNNs. WikiArt's 81k images with 195 artist classes would likely overfit.
- **CRNN (our approach)**: CNN extracts spatial features, RNN captures sequential patterns across canvas columns -- best fit for style where texture repetition matters.

Why Multi-Task over Separate Models?

- Shared backbone learns **general visual features** that benefit all tasks
- Genre (general: 10 classes) acts as a **regularizer** for the more specific heads
- 3x parameter efficiency -- one backbone instead of three
- Style <-> Artist correlation provides **implicit data augmentation** (knowing an artist helps predict style)

General vs. Specific Hierarchy

Level	Task	Classes	Head Design	Metric
General	Genre	10	Small FC + Weighted XEnt	Weighted F1
Specific	Style	27	BiGRU + Weighted XEnt	Weighted F1
Most Specific	Artist	195	FC + Label Smoothing	Top-5 Acc + Weighted F1

Multi-Task Loss Weights

Ls=1.0 (primary task), La=0.8, Lg=0.8. Style weighted highest because it is the core task; artist and genre provide supporting signal without dominating gradients.

Evaluation

- **3 separate Weighted F1 scores** (one per task) -- each task has its own label space
- **Average F1** across all three tasks as a single summary metric
- **Top-5 accuracy** for Artist since many artists share visual similarity
- Per-class classification reports for Style and Genre